SEUPD@CLEF: Team DSPY

Carlo Galiano[1], Alberto Iurlo[1] and Stefano Riato[1]

[1]University of Padua, Italy

Abstract

This paper presents Team DSPY's Information Retrieval (IR) system for the CLEF 2026 CheckThat! Lab Task 1, where the main goal is to retrieve scientific publications for multilingual social media claims. The primary challenge is the high degree of vocabulary mismatch between short multilingual tweets and long technical abstracts centered on English. As such, the development of our system goes beyond the initial baseline algorithm (i.e. BM25) to a sophisticated neural architecture. As the results of empirical analysis suggested that statistical query expansion (Pseudo-Relevance Feedback) suffers from topic drift, we designed an **Asymmetric Hybrid Search Pipeline**. This novel approach routes English queries through a highly-tuned lexical engine, while reserving computationally expensive long-context dense retrieval (BGE-M3) and Cross-Encoder semantic re-ranking strictly for the processing of French and German queries. The proposed solution has proved to be efficient in terms of the averaged MRR@5 score.

Keywords

Multilingual Information Retrieval, Claim Verification, Asymmetric Hybrid Search, Dense Retrieval, Cross-Encoder Re-ranking, Vocabulary Mismatch, CLEF CheckThat! Lab

1. Introduction

This report aims to provide a brief explanation of the Information Retrieval system built for Task 1. This paper aims to share the experiments, the analysis, and the results obtained over the course of one month in order to compete in the CheckThat! Lab Task 1 competition.

The CLEF 2026 CheckThat! Lab Task 1 frames scientific claim retrieval as a low-resource, cross-lingual ad-hoc retrieval problem Hafid et al. [1]. As shown in Figure 1, given a social media post (e.g. tweet) that implicitly cites a scientific paper, mainly in English, German, or French, containing opinions, comments, hashtags, or tags, the system must rank a fixed corpus of 10,000 scientific papers so that the cited paper appears as high as possible in the returned list. The task is challenging for two reasons: (i) tweets are short, noisy, and multilingual, while scientific abstracts are long, technical, and predominantly written in English; and (ii) a German or French query may correspond to an English target paper, so monolingual retrieval breaks the assumption that query and document share a vocabulary.

We approach this problem as an opportunity to systematically apply and evaluate core IR techniques — from sparse lexical models to dense neural retrieval — against a real-world competitive benchmark. Our experiments investigate the trade-offs between retrieval effectiveness and computational efficiency under the constraints imposed by the competition setting.

The paper is organized as follows: Section 2 describes our approach; Section 3 explains our experimental setup; Section 4 discusses our main findings and problems; finally, Section 5 draws some conclusions and outlooks for future work.

2. Methodology

We developed these pipelines based on the knowledge gained during the course. This section describes the indexing and retrieval pipeline. We first present the BM25 ranking model that underpins every

"Search Engines", course at the master degree in "Computer Engineering", Department of Information Engineering, and at the master degree in "Data Science", Department of Mathematics "Tullio Levi-Civita", University of Padua, Italy. Academic Year 2025/2026

carlo.galiano.1@studenti.unipd.it (C. Galiano); alberto.iurlo@studenti.unipd.it (A. Iurlo); stefano.riato@studenti.unipd.it (S. Riato)

© 2026 Copyright for this paper by its authors.
Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

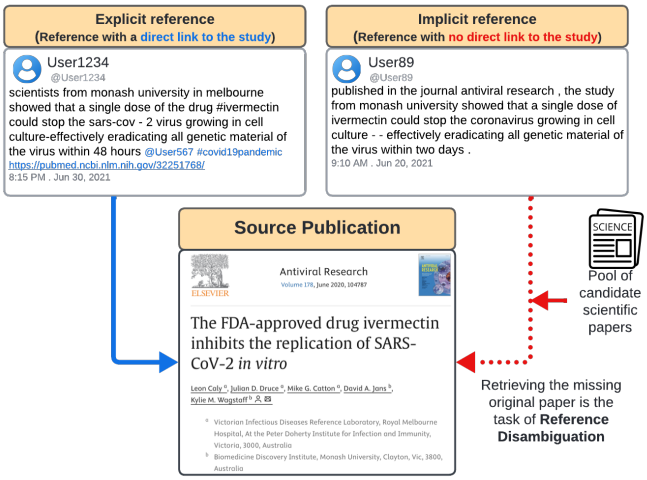


Figure 1: Source Retrieval for Scientific Web Claims problem [Public Domain], it shows a claim with an implicit reference for which the source publication (box to the right) needs to be retrieved in task 4b (adapted from Hafid et al. [1])

version (§2.1), then the document and query parsers (§2.2), the linguistic preprocessing (§2.3), and the indexing layer (§2.4). The four iterative versions (V1 – V4) are described in §2.5 – §2.8.

2.1. BM25

Every version uses BM25 [2, 3], a probabilistic ranking function that extends the Binary Independence Model [4] with two components that the BIM lacks: *Term Frequency (TF)* saturation and document-length normalisation. The full ranking function is

$$R(q,d) \;\propto\; \sum_{i \in q \cap d} \underbrace{\log \frac{(r_i + 0.5)\,(N - n_i - R + r_i + 0.5)}{(n_i - r_i + 0.5)\,(R - r_i + 0.5)}}_{\text{IDF-like weight (BIM)}} \cdot \underbrace{\frac{(k_1 + 1)\, f_i}{k_1\!\left(1 - b + b\,\frac{dl}{avdl}\right) + f_i}}_{\text{TF saturation}} \cdot \underbrace{\frac{(k_2 + 1)\, qf_i}{k_2 + qf_i}}_{\text{query TF}} \tag{1}$$

where N is the number of documents in the collection; n_i is the document frequency of term i; R and r_i are relevance-feedback counts (zero in the absence of feedback); f_i is the within-document frequency of term i; qf_i is its frequency in the query; dl is the document length in tokens; $avdl$ is the mean document length across the collection; and k_1, b, k_2 are free parameters.

TF saturation (k_1). The TF component is a sigmoid-like function of f_i that saturates as f_i grows, approaching $(k_1 + 1)/k_1$ asymptotically. This models the intuition that the first occurrence of a term is highly informative whereas the hundredth adds negligible evidence. At $k_1 = 0$ the function reduces to pure IDF; as $k_1 \to \infty$ it degenerates to raw, unsaturated TF.

Length normalisation (b). The factor $\left(1 - b + b\,dl/avdl\right)$ penalises long documents, which naturally accumulate higher raw term frequencies regardless of relevance. Setting $b = 0$ disables normalisation entirely; $b = 1$ applies full normalisation by dividing TF by the relative document length.

Query TF (k_2). For standard keyword queries where each term appears exactly once ($qf_i = 1$), the query-TF component reduces to 1 and has no effect. It becomes relevant only when the query is itself derived from a document containing repeated terms.

2.2. Document and Query Parsing

Documents and queries are parsed through dedicated classes that implement the Python iterator protocol, so the system processes the files sequentially without loading the whole collection into memory.

2.2.1. Document Parsing

The `JsonArrayDocumentParser` reads the corpus in batches of 500 JSONL records. For each record, the title and abstract are concatenated and passed to the `langdetect` library [5] to obtain a language code. Codes outside $\{\texttt{en}, \texttt{fr}, \texttt{de}\}$, together with inputs that raise `LangDetectException` on very short or noisy text, are mapped to a single `other` bucket. Determinism is enforced by setting `DetectorFactory.seed = 0`, so repeated runs over the same collection produce identical labels. Each parsed record is wrapped in a `ParsedDocument` carrying the document identifier, title, abstract, and detected language; the language label is later consumed by the indexer to route the document to the appropriate per-field analyser (§2.4).

2.2.2. Query Parsing

Queries are parsed by `JsonArrayQueryParser`, mirroring the batched-iterator pattern of the document parser. Because the queries are social-media tweets rather than scientific articles, a cleaning step is applied before any retrieval logic: user mentions (`@handle`) are stripped and the hashtag character is removed but the term itself is preserved, the leading numeric prefix of enumerated tweets (e.g. `1)`, `2/`) is dropped, and the Lucene-reserved tokens AND/OR/NOT are lowercased so that an upper-case sentence opener does not crash the query parser. Alphanumeric characters and a small set of essential punctuation (period, comma, hyphen, parentheses, percent sign) are kept; everything else is removed.

Unlike documents, queries are not passed through `langdetect`: the dataset partitions tweets per language *a priori*, and the language code is read directly from the file name (`en_dev.jsonl`, `de_dev.jsonl`, `fr_dev.jsonl`).

2.3. Linguistic Preprocessing

All linguistic preprocessing, for both documents and queries, is delegated to Apache Lucene analysers [6], wrapped in a `PerFieldAnalyzerWrapper` so that each (title/abstract, language) field receives its own analysis chain:

- `EnglishAnalyzer` — Unicode tokenisation, lowercase conversion, English possessive stripping, English stopword removal, and Porter stemming [7].
- `FrenchAnalyzer` — elision filtering (*l'*, *d'*, …), lowercase conversion, French stopword removal, and the French light stemmer.
- `GermanAnalyzer` — lowercase conversion, German stopword removal, normalisation of umlauts and *ß*, and the German light stemmer.
- `StandardAnalyzer` — Unicode-aware tokenisation, lowercase conversion, and removal of the default English stopword set; used as a typed-but-language-agnostic fallback for documents whose detected language is neither English, French, nor German (e.g. Italian or Spanish).

Using the same analyser at both index time and query time ensures that identical surface forms map to identical indexed terms on both sides of the inverted-index lookup, which is essential for stemmed languages such as German where the surface form and the stem can differ substantially.

Discarded alternative: a hand-rolled NLTK pipeline. An earlier preprocessing variant relied on a custom `TextProcessor` class layered on top of NLTK [8]. HTML tags were stripped with `re.sub(r'<[^>]+>', '', text)`; tokens were extracted with to retain alphanumerics, percentages and hyphenated compounds such as *SARS-CoV-2*; stopwords were removed with NLTK's per-language

lists; and stems were produced by NLTK's Snowball implementation [9]. We abandoned this pipeline for two reasons. First, executing the analysis in Python at index time and again at query time made it difficult to guarantee bit-identical token streams on the two sides, risking subtle index/query mismatches. Second, in pilot runs against the dev split it was strictly less effective than the Lucene-analyser configuration above, while introducing an additional external dependency.

2.4. Indexing

The collection is indexed by `DirectoryIndexer`, a Python class that walks the corpus directory and feeds qualifying files through `JsonArrayDocumentParser`. A whitelist parameter (`include_files = {collection.jsonl}`) restricts the walk to the document file proper, so the topic and query files which share the same folder and `.jsonl` extension are skipped rather than mis-indexed as documents.

2.4.1. Field schema

Each `ParsedDocument` is converted into a Lucene `Document` carrying four fields:

- `id` — `StringField` (not analysed, stored). Persists the original `pubkey` so retrieved hits can be written into the TREC run file.
- `lang` — `StringField` (not analysed, stored). Persists the language label assigned at parse time.
- `title_⟨lang⟩` and `abstract_⟨lang⟩` — typed text fields carrying the document body. The language suffix routes each document to the analyser configured for its language; e.g. a record labelled `de` populates only `title_de` and `abstract_de`, leaving the remaining language fields empty. This per-language fan-out lets the English-, French-, and German-specific analysis chains coexist inside a single index.

The text fields share a singleton `FieldType` configured with `DOCS_AND_FREQS_AND_POSITIONS`, tokenisation enabled, and the original text stored. Storing positions enlarges the index by approximately 15–20 % but is required to support phrase and proximity queries (*"machine learning"*); storing the raw text supports stored-field retrieval in the cross-encoder re-ranking stages of V3 and V4.

2.4.2. Scoring and writer configuration

The `IndexWriter` is configured with BM25 similarity ($k_1 = 0.9$, $b = 0.8$; see §2.1), a 256-MB RAM buffer, `OpenMode.CREATE` (so each indexing run starts from scratch), `commitOnClose=true`, and compound files disabled to keep segment inspection straightforward. Indexing the 10,000-document collection completed in under one minute on the Docker-hosted CPU configuration described in §3.

2.4.3. Dense companion index

For the hybrid pipelines (V3, V4) a second indexer in `index_faiss.py` runs alongside the Lucene indexer and produces a FAISS-backed dense index [10] over the same collection. Each title and abstract is concatenated into a single `title. abstract` string, embedded by a sentence-transformer encoder (`BAAI/bge-m3` [11] for V4, `intfloat/multilingual-e5-base` [12] for V3, configurable via the `FAISS_MODEL` environment variable), ℓ_2-normalised, and added to a flat inner-product index (`faiss.IndexFlatIP`). Because the vectors are ℓ_2-normalised, inner-product search on this index is mathematically equivalent to cosine-similarity search. Two artefacts are written next to the Lucene segments: `corpus.faiss` (the serialised FAISS index) and `doc_ids.json` (the parallel list of document identifiers needed to map FAISS row indices back to Lucene's `id` field). Embedding is performed in batches of four documents to bound peak GPU memory at the precision (FP16) and dimensionality (1024 for bge-m3) used in V4.

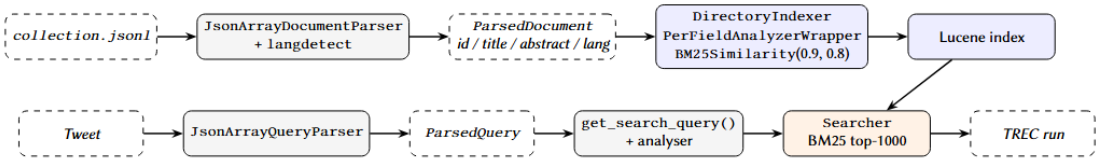


Figure 2: V1 — full pipeline. *Top:* offline corpus path (responsible: `index.py`), `collection.jsonl` → `JsonArrayDocumentParser` (with deterministic `langdetect`) → `ParsedDocument` → `DirectoryIndexer` (per-field analysers, BM25 scoring) → a single Lucene index. *Bottom:* online query path (responsible: `search.py`), tweet → `JsonArrayQueryParser` → `ParsedQuery` → tweet cleaning + the same per-field analyser → BM25 search. The Lucene index links the two halves of the pipeline.

2.5. V1: Lexical Baseline and Probabilistic Retrieval

The foundational pipeline (V1, Figure 2) relied exclusively on exact keyword matching via Apache Lucene and the BM25 probabilistic scoring model.

Configuration. We set $k_1 = 0.9$ and $b = 0.8$, departing slightly from the commonly reported defaults of 1.2 and 0.75. The lower k_1 accelerates TF saturation, reducing the influence of high-frequency terms in scientific abstracts (e.g. domain-specific verbs that recur heavily but carry little discriminative power). The higher b strengthens length normalisation, which is appropriate for a collection in which document length varies considerably between short news-style abstracts and full structured abstracts. The query-TF parameter k_2 is not exposed by Lucene's `BM25Similarity` implementation; the component is effectively fixed at $k_2 \to \infty$, giving raw query TF, but since every query term appears at most once in our setting this has no practical effect.

Multilingual Indexing. To preserve complex medical terminology and hyphenated compounds, we indexed documents using native language-specific analysers (`EnglishAnalyzer`, `FrenchAnalyzer`, `GermanAnalyzer`) rather than a generic tokeniser; an early NLTK-based variant was discarded for the reasons given in §2.3. BM25 scoring is computed by Apache Lucene's `BM25Similarity`, which applies the per-field language-specific analysers (English, French, German, and a `StandardAnalyzer` fallback for other languages) before indexing and query processing, ensuring that stemming and stopword removal operate in the correct morphological context for each field.

The Bottleneck. While this baseline performs well for English queries, where the vocabulary is standardised, it suffers from a substantial vocabulary mismatch on cross-lingual tasks (French and German). Lexical models fail when query terms lack exact character-level translations in the target documents, motivating the experiments that follow.

2.6. V2: Statistical Query Expansion via Pseudo-Relevance Feedback

To bridge the vocabulary gap without introducing external neural models, the second pipeline (V2, Figure 3) explored a statistical expansion technique known as Pseudo-Relevance Feedback [13].

Mechanism. PRF assumes that the initial top-k retrieved documents are highly relevant. The system retrieves the top-3 documents from a first BM25 pass, merges their multilingual title and abstract fields, and selects the five most frequent content terms after stopword removal against a curated multilingual list. These terms are then appended to the original query as a `SHOULD` clause to expand the search vocabulary while the original query is kept as a `MUST` constraint.

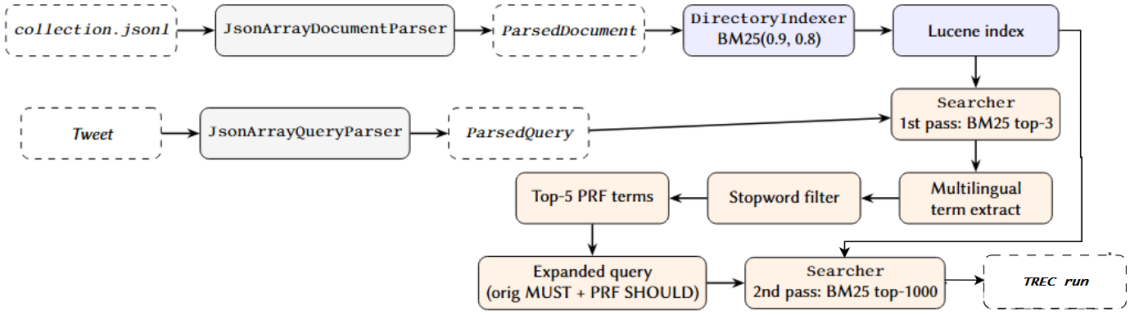


Figure 3: V2 — full pipeline. Same offline indexing as V1; the online query path adds an intermediate BM25 pass over the top-3 documents to extract the five most frequent multilingual content words, which are appended as a SHOULD clause to the original query before a second BM25 pass.

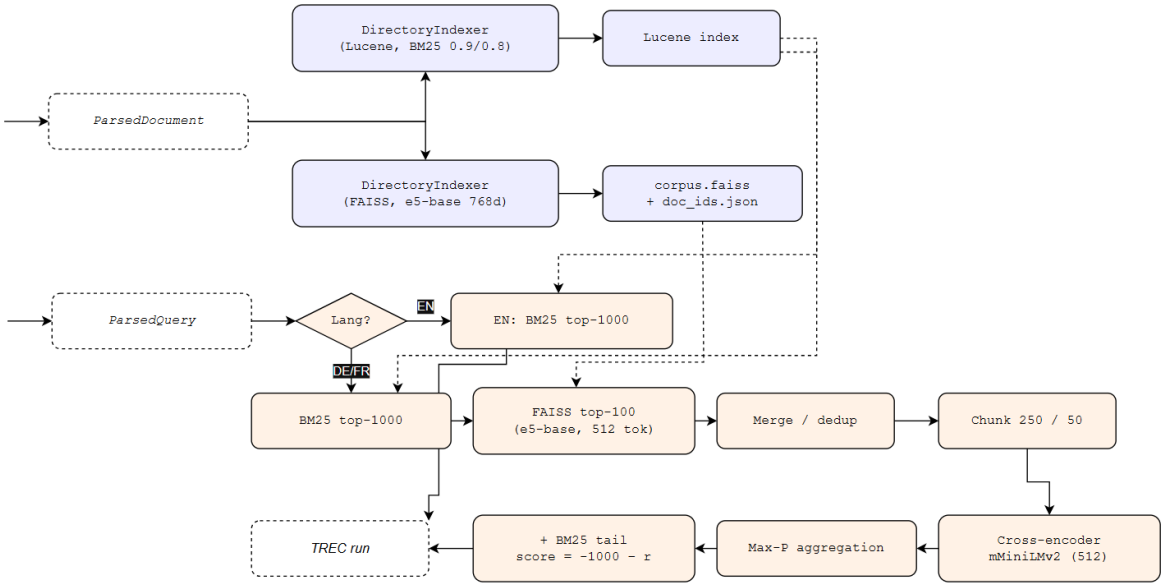


Figure 4: V3 — full pipeline. Two parallel `DirectoryIndexer` runs over the same `ParsedDocument` stream produce a Lucene inverted index and a FAISS dense index (`e5-base`, ℓ_2-normalised) sharing identifiers via `doc_ids.json`. English queries follow a fast BM25-only path; French and German queries fuse BM25 and dense candidates, chunk long abstracts to fit the 512-token cross-encoder, and aggregate per document via Max-P. The BM25 tail pads the positions beyond the rerank pool.

Theoretical Limitation. As discussed in §4, the technique is highly susceptible to topic drift. When the initial retrieval is even marginally inaccurate, the statistically extracted terms shift the semantic focus away from the user's intent. Given the high precision required in the medical domain, this purely statistical expansion proved unsuitable.

2.7. V3: Hybrid Asymmetric Search and Semantic Re-ranking

Drawing on the limits of the previous pipelines, the third experiment (V3, Figure 4) moved from statistical expansion to a deep-learning approach intended to capture semantic similarity through embeddings. We adopted an asymmetric pipeline that combines high-precision re-ranking with computational efficiency.

Hybrid Candidate Pooling. To resolve the retrieval bottleneck identified in §2.5 — relevant cross-lingual documents that BM25 fails to retrieve — we implemented dense retrieval over FAISS [10]. For French and German queries, the system merges BM25 candidates with those from a dense search

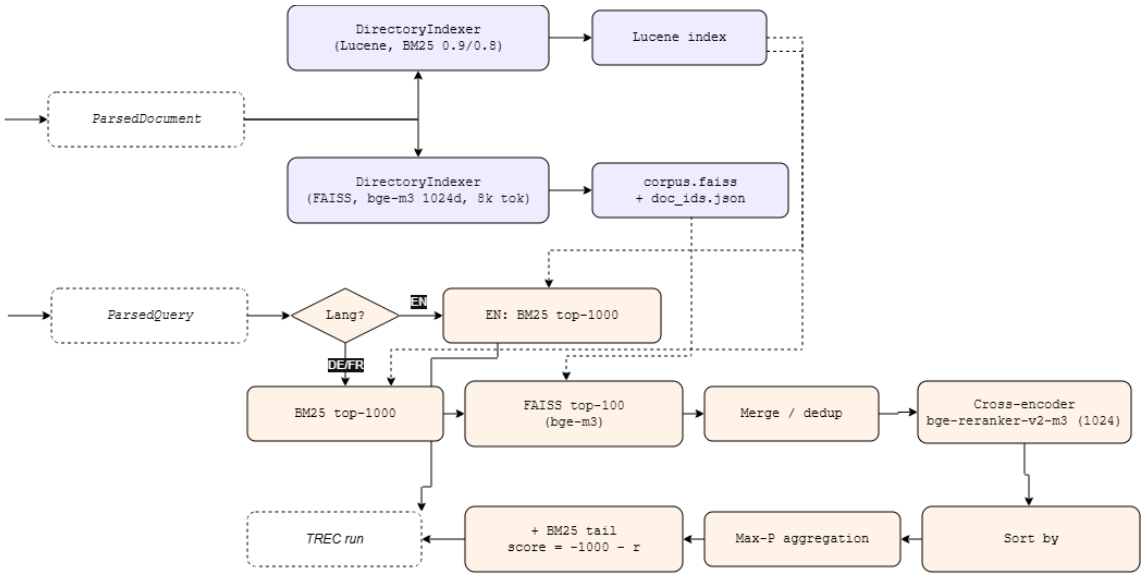


Figure 5: V4 — full pipeline. Indexing layout matches V3, but the FAISS side now stores 1024-dimensional bge-m3 embeddings. At query time the chunking and Max-P stages of V3 are removed: the upgraded cross-encoder `bge-reranker-v2-m3` accepts up to 1024 tokens and accommodates nearly all abstracts in a single forward pass.

that uses the `intfloat/multilingual-e5-base` encoder [12], recovering documents that share meaning with the query but have no surface-vocabulary overlap.

Neural Re-ranking (Cross-Encoder). Scoring on the merged candidate pool is performed by a Transformer-based cross-encoder (`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`) [14, 15]. Unlike the bi-encoder used for first-stage dense retrieval, which encodes query and document independently, the cross-encoder concatenates them and applies full self-attention, allowing deep word-level semantic interactions that are critical for cross-lingual understanding.

Max-P Chunking Strategy. To respect the 512-token context limit of this cross-encoder while preserving the information in longer abstracts, we adopted a sliding-window strategy. Documents are partitioned into chunks of 250 words with a 50-word overlap; each chunk is scored independently, and the document's final score is the maximum across its chunks (*Max-P* aggregation [16]).

Asymmetric Optimisation. Given the existing strength of the lexical engine on English, English queries bypass the neural pipeline and follow a fast BM25 path. This preserves computational budget for the harder French and German cross-lingual tasks.

Tail Padding Integrity. To preserve the standard Recall@1000, documents retrieved by BM25 but not selected for re-ranking are appended to the final results. These tail documents receive penalised scores using the formula $score = -1000 - rank$, which prevents the score-based sort used by evaluation toolkits from promoting tail entries above the cross-encoder's reranked head.

2.8. V4: Architecture with Long-Context Embeddings

Although V3 successfully addressed the cross-lingual vocabulary gap, the chunking strategy added engineering complexity and risked breaking long-range semantic dependencies. The final iteration (V4, Figure 5) optimised the hybrid pipeline by upgrading both neural components to long-context, retrieval-tuned state-of-the-art models.

Long-Context Bi-Encoder (BGE-M3). We replaced the dense encoder with `BAAI/bge-m3` [11], a retrieval-tuned multilingual model with a context window of up to 8192 tokens. The window is far larger than any abstract in the collection, so each document is now encoded in a single pass on the FAISS side, removing both information loss and the architectural complexity of the sliding-window mechanism.

Long-Context Cross-Encoder (BGE-Reranker-v2-m3). The cross-encoder was upgraded to `BAAI/bge-reranker-v2-m3` [11], which accepts up to 1024 tokens per (query, document) pair, twice the budget of the V3 reranker. Because nearly all abstracts fit within 1024 reranker tokens, V4 removes the chunking and Max-P stages of V3 entirely. The chunking removal is therefore driven by the *cross-encoder* upgrade, not by the bi-encoder's 8192-token window, which only governs first-stage dense retrieval.

Unified High-Dimensional Space. The dense embedding dimensionality grew from 768 (e5-base) to 1024 (bge-m3), giving the first dense-retrieval stage a richer semantic representation and a higher-quality candidate pool for the re-ranker on French and German queries.

Refined Hybrid Integrity. Asymmetric routing is preserved in V4 along with the tail-padding strategy introduced in V3. The combination of long-context dense retrieval, exact-match lexical backup, and protected evaluation integrity constitutes the most robust configuration we evaluated for cross-lingual scientific-claim retrieval.

2.9. Embeddings

The retrieval pipelines exploit two complementary families of embeddings. *Sparse* embeddings are high-dimensional vectors in which the majority of entries are zero — concretely, the inverted-index posting lists exposed by Lucene. They are computationally lightweight and excel at exact-match retrieval, but do not encode sentence-level meaning. *Dense* embeddings are low-dimensional, contextual vectors in which each dimension carries a fragment of semantic meaning, computed by transformer encoders trained for retrieval (`multilingual-e5-base` in V3, `bge-m3` in V4). Because dense vectors place semantically similar documents close together regardless of surface form, they recover relevant documents that lexical retrieval misses, narrowing the cross-lingual vocabulary gap that motivates V3 and V4.

3. Experimental Setup

3.1. Document Collection and Queries

Document corpus. The document collection consists of 10,000 scientific publications provided by the CLEF 2026 CheckThat! Lab Task 1 organisers. Each publication is described by the following metadata fields: `title`, `abstract`, `authors`, `venue`, and a unique identifier `pubkey`. In our system, we index only the `title` and `abstract` fields, as these carry the core semantic content of a paper and are available for all documents in the corpus.

Query sets. The queries consist of short social media posts (tweets), partitioned into three official splits: training, development, and test. Each query is provided as a `text` field containing the raw social media content, along with a `pubkey` field that identifies the single relevant document in the corpus. This yields a *one-to-one relevance mapping*: each query is associated with exactly one ground-truth document, making this a highly precise but recall-agnostic retrieval setting.

Table 1 summarises the statistics of the collection and query splits.

Table 1
Collection and query statistics.

Split	English	German	French
Train queries	14977	1460	2807
Dev queries	3905	386	702
Test queries	6080	876	1220
Corpus documents		10,000	

3.2. Evaluation Measures

We evaluate retrieval quality using the official metric *Mean Reciprocal Rank (MRR)*@5, which rewards systems that place the single relevant document in the top-5 ranked positions. In addition, we report *Mean Average Precision (MAP)*, *Normalized Discounted Cumulative Gain (NDCG)*@10, Precision@1, and Recall@1000 to characterise different aspects of ranking quality. Evaluation is performed with the `ranx` library using relevance judgements (`qrels`) derived from the ground-truth `pubkey` labels provided with each query.

3.3. Repository

The source code is publicly available at: https://bitbucket.org/upd-dei-stud-prj/seupd2526-dspy/src/master/. The repository is organised as follows:

- `pylucene/src/` — entry point orchestrator (`runner.py`) and data acquisition scripts.
- `pylucene/src/Index/` — core PyLucene and FAISS indexing pipelines (`index.py`, `index_faiss.py`).
- `pylucene/src/Search/` — core execution scripts for the four retrieval methodologies (`search_v1.py` to `search_v4.py`).
- `pylucene/src/Parse/` — data pre-processing and formatting classes for documents and queries.
- `pylucene/src/Evaluation/` & `utils/` — scoring (`evaluate.py`), report generation, ground truth creation, and submission formatting.
- `experiment/` — generated artefacts (index, runs, qrels); not committed to version control.

3.4. Hardware and Software

Experiments were conducted on three machines. All ran the retrieval pipeline inside a Docker container built from the official `coady/pylucene` image, which bundles PyLucene 9 (Lucene 9.x) on Python 3.11 (to check).

Table 2
Hardware specifications of the development machines.

Component	Machine 1	Machine 2	Machine 3
CPU	Intel Core i5-13420H (13th Gen)	AMD Ryzen AI 7 350	Intel Core i7-6500U @ 2.50 GHz
RAM	32 GB DDR5 5600 MHz	16 GB DDR5 5600 MHz	12 GB DDR4 2133 MHz
GPU	Intel UHD Graphics	AMD Radeon 860M + NVIDIA RTX 5060 Laptop	NVIDIA GeForce 920MX
OS	Windows 11 Home	Windows 11 Home	Windows 10 Home

The Docker environment also exposes a Jupyter notebook server on port 8888 for Exploratory Data Analysis. The GPU of Machine 2 was used to accelerate the neural cross-encoder re-ranker (`rerank.py`) and FAISS dense retrieval experiments (`index_faiss.py` and `search_faiss.py`). All BM25 indexing

, and *Pseudo-Relevance Feedback (PRF)* are fully reproducible on *Central Processing Unit (CPU)* and were run on all three machines.

4. Experimental Results

This section presents the quantitative evaluation of the four runs in the development and training phases. The results highlight how each architectural decision was motivated by the limitations witnessed in the previous system.

4.1. V1 — BM25 Baseline

V1 is the starting reference point for all subsequent runs. As shown in Table 3, it achieves an average MRR@5 of 0.2190, a MAP of 0.2322, and an NDCG@10 of 0.2488 on the development set. The per-language division in Table 4 displays a strong imbalance among the languages: English reaches MRR@5 = 0.4910, while German and French respectively have the scores 0.0968 and 0.0691. This huge gap suggests that the baseline is competitive for English, but much less effective in the multilingual setting addressed by the task. In the training set, the same pattern remains visible. Table 5 shows that V1 reaches an average MRR@5 of 0.2301, MAP of 0.2437, and NDCG@10 of 0.2606, while Table 6 reports MRR@5 values of 0.5028 for English, 0.0970 for German, and 0.0904 for French. The consistency between development and training results highlights that the main limitation of V1 is structural rather than split-specific, and therefore motivates the search for a system that can improve non English query retrieval.

4.2. V2 — Pseudo-Relevance Feedback

V2 was the first system developed as an attempt to improve the baseline retrieval , but as previously anticipated, the results show a consistent deterioration. In the development set, Table 3 shows that average MRR@5 decreases from 0.2190 in V1 to 0.1892 in V2, while MAP drops from 0.2322 to 0.2011 and NDCG@10 from 0.2488 to 0.2185. The same trend repeated itself in the training set, where average MRR@5 falls from 0.2301 to 0.1971, MAP from 0.2437 to 0.2102, and NDCG@10 from 0.2606 to 0.2284 (Table 5).
The per language results confirm that the decrease is not attributable to a single language. In the development set (Table 4), English MRR@5 goes down from 0.4910 to 0.4285, German from 0.0968 to 0.0806, and French from 0.0691 to 0.0585. A similar pattern is observed in the training set (Table 6), where all three language specific values remain below those of V1. Since V2 metrics are inferior with respect to the baseline, it was necessary to continue the search of a better performing model.

4.3. V3 — Hybrid Asymmetric Search

V3 represents the first substantial improvement over the baseline system. In the development set, Table 3 shows that average MRR@5 increases to 0.3913, with MAP reaching 0.4054 and NDCG@10 rising to 0.4277. Compared to V1 and V2, this is a clear breakthrough in all three metrics. The same result is confirmed by the training set, where V3 achieves the best scores among the evaluated runs: MRR@5 = 0.3957, MAP = 0.4103, and NDCG@10 = 0.4318 (Table 5). The language results clarify where this achievement comes from, bringing to light that the gain is completely driven by the hybrid dense-sparse retrieval introduced. In the development set (Table 4), English matches V1 at 0.4910, while German and French improve significantly from 0.0968 to 0.2919 and from 0.0691 to 0.3910, respectively. The same behavior is observable in the training set (Table 6), where English remains at 0.5028, while German increases to 0.2906 and French to 0.3937. The results obtained show that the new formulation highly improves the retrieval over German and French, achieving an overall better performance. Nevertheless, the gap between English and the other two languages suggested that the Max-P chunking strategy used

was causing the fragmentation of the long semantic dependencies, justifying the search for another model.

4.4. V4 — SOTA Hybrid Architecture

V4 achieves the best results on the development set and represents the final retrieval system formulation. As shown in Table 3, it reaches an average MRR@5 of 0.4919, a MAP of 0.5038, and an NDCG@10 of 0.5298, outperforming all previous runs on every metric. In detail, the jump from V3 to V4 is remarkable, approximately gaining 10 percentage points over the three metrics. The per language results in Table 4 show that V4 is also the most balanced system among the languages. English decreases a bit to 0.4898, which is really close to the best previous English results, while German increases to 0.4382 and French to 0.5476. Compared to V3, this means a substantial gain from 0.2919 to 0.4382 in German and from 0.3910 to 0.5476 in French. These results confirm that switching to `BAAI/bge-m3` and eliminating the Max-P chunking strategy (Section 2.8) was the key architectural decision to overcome the weakness related to the metrics' gap among the languages.

4.5. Results Tables

Table 3: Averaged results on the development set.

Run	MRR@5	MAP	NDCG@10
V4 — SOTA Hybrid (bge-m3 + CE)	**0.4919**	**0.5038**	**0.5298**
V3 — Hybrid Asymmetric (e5 + CE)	0.3913	0.4054	0.4277
V1 — BM25 Baseline	0.2190	0.2322	0.2488
V2 — BM25 + PRF	0.1892	0.2011	0.2185

Table 4: Per-language metrics result in the development set.

Run	EN	DE	FR	Avg
V4 — SOTA Hybrid (bge-m3 + CE)	0.4898	**0.4382**	**0.5476**	**0.4919**
V3 — Hybrid Asymmetric (e5 + CE)	**0.4910**	0.2919	0.3910	0.3913
V1 — BM25 Baseline	**0.4910**	0.0968	0.0691	0.2190
V2 — BM25 + PRF	0.4285	0.0806	0.0585	0.1892

Table 5: Averaged results on the training set.

Run	MRR@5	MAP	NDCG@10
V3 — Hybrid Asymmetric (e5 + CE)	**0.3957**	**0.4103**	**0.4318**
V1 — BM25 Baseline	0.2301	0.2437	0.2606
V2 — BM25 + PRF	0.1971	0.2102	0.2284

Table 6: Per-language metrics result in the training set

Run	EN	DE	FR	Avg
V3 — Hybrid Asymmetric (e5 + CE)	**0.5028**	**0.2906**	**0.3937**	**0.3957**
V1 — BM25 Baseline	**0.5028**	0.0970	0.0904	0.2301
V2 — BM25 + PRF	0.4377	0.0795	0.0742	0.1971

5. Conclusions and Future Work

5.1. Conclusions

This paper presented four retrieval systems developed for the Scientific Retrieval task proposed by CLEF, addressing the challenge of matching informal social media queries to multilingual scientific documents in English, French, and German. Our experiments expose a consistent progression in retrieval achievement (measured as scores of the metrics used) across the runs. The baseline, while on one hand established the upper bound for English queries, on the other hand highlighted the structural limits of an exact matching system based; indeed, the vocabulary mismatch between informal social media posts and scientific documents caused the collapse of the metrics scores for German and French. The attempt to address this gap through the query expansion technique Pseudo Relevance Feedback has further decreased the scores, confirming the poor quality of the initial retrieval set. The introduction of hybrid dense-sparse retrieval through FAISS, combined with a Cross-Encoder re-ranker, produced a first substantial improvement in the retrieval of non English queries; indeed, the deep neural networks used for German and French allowed the retrieval of documents thanks to the semantic embeddings similarity. The final architecture refined the hybrid pipeline by using a more powerful long-context retrieval model, which eliminated the previous chunking need. The results achieved were the highest and most balanced across all three languages, confirming that the hybrid dense-sparse retrieval with a neural re-ranking represents the most effective architecture for multilingual scientific document retrieval in this setting.

5.2. Future Work

The results obtained in this work highlight and suggest several directions that could improve retrieval quality and system design.
The different pipelines demonstrated the huge contribution of semantic retrieval to query performance. A natural follow up would be to investigate more specifically natural language processing strategies in order to adapt the search to the data nature.
A deliberate structural choice throughout our experiments was to avoid translation, since we were concerned about introducing noise or losing key terminology fundamental for the retrieval process. However, our results suggest that this strategy should be tested: large language models could be exploited for translation. Moreover, witnessing the failure of PRF strategy, LLMs could also be used for modern query expansion techniques.
From a software engineering perspective, our development process revealed a significant technical debt: each system version was implemented as an independent pipeline, which on one hand reduced the time required to implement each run, but on the other hand made it harder to explore alternatives and collaborate among the team members. Future work should prioritize the design of an architecture within which main components are defined as a class, making it easier to swap them inside a unique pipeline. This would reduce the cost, both in terms of building and running time, enabling a more practical work environment.

6. Group Members Contribution

All the members contributed based on the capability and comprehension of the topic. Thanks to the different backgrounds (Business Management and Informatics, Statistics and Math) each of the member could contribute in different part of the project. Below is a summary of the main contributions made by each participant, based on the Git commit history:

Carlo Galiano helped with the initial infrastructure setup, developing the dataset acquisition scripts and the data parsers with language detection capabilities (langdetect library). He engineered the core retrieval pipelines, starting from the multi-field BM25 lexical baseline with tuned hyperparameters and native multilingual analyzers, up to Cross-Encoder semantic re-ranking. Furthermore, he implemented the evaluation framework using the `ranx` library, developed the automated user-friendly script (`runner.py`), and contributed to the drafting of the Methodology sections of this paper.

Alberto Iurlo On the implementation side, the contribution consisted in: document parsing, text processing, boosting different query fields, bug fixing in the retrieval pipeline, and implementation of the FAISS index and search component for dense retrieval. On the report side, the contribution consisted in: helping with methodologies, writing experimental results, conclusions and future works.

Stefano Riato prepared the documentation on how to use GIT and Docker for supporting the team in following specific rules and be ready to contribute to the project. Refactoring of indexing and search pipelines of PyLucene to support multi-language queries, PRF integration, and optional FAISS vector-based search. Optimization of document and query parsers through Abstract Base Classes and deque implementations for improved memory efficiency. Development of corpus validation routines and resolution of critical bugs in query parsing and leaderboard scoring. Management of Docker environment, project layout reorganization. Authorship of Introduction, Methodology, Set up and BIM theory sections in the technical report, including diagram creation for each version.

References

[1] S. Hafid, Y. S. Kartal, S. Schellhammer, V. Jacot, S. Bringay, S. Dietze, K. Todorov, Disambiguation of implicit scientific references on x, in: Proceedings of the 36th ACM Conference on Hypertext and Social Media, ACM, 2025.

[2] D. K. Harman, Overview of the Third Text REtrieval Conference (TREC-3), ????, pp. 1–19.

[3] S. E. Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu, M. Gatford, Okapi at TREC–3, ????, pp. 109–126.

[4] S. E. Robertson, K. Spärck Jones, Relevance Weighting of Search Terms, Journal of the American Society for Information Science (JASIS) 27 (1976) 129–146.

[5] N. Shuyo, Language detection library for java (2010). URL: https://github.com/shuyo/language-detection, accessed: 2026-04-30.

[6] M. McCandless, E. Hatcher, O. Gospodnetić, Lucene in Action, 2nd ed., Manning Publications Co., NY, USA, 2010.

[7] M. F. Porter, An algorithm for suffix stripping, Program 14 (1980) 130–137.

[8] S. Bird, E. Loper, NLTK: The natural language toolkit, in: Proceedings of the ACL 2004 on Interactive Poster and Demonstration Sessions, Association for Computational Linguistics, 2004, pp. 31:1–31:4.

[9] M. F. Porter, Snowball: A language for stemming algorithms, 2001. URL: https://snowballstem.org/, accessed: 2026-04-30.

[10] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (2021) 535–547.

[11] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, arXiv preprint arXiv:2402.03216 (2024).

[12] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei, Multilingual E5 text embeddings: A technical report, arXiv preprint arXiv:2402.05672 (2024).

[13] V. Lavrenko, W. B. Croft, Relevance-based language models, in: Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), 2001, pp. 120–127.
[14] L. H. Bonifacio, V. Jeronymo, H. Q. Abonizio, I. Campiotti, M. Fadaee, R. Lotufo, R. Nogueira, mMARCO: A multilingual version of the MS MARCO passage ranking dataset, in: arXiv preprint arXiv:2108.13897, 2021.
[15] R. Nogueira, K. Cho, Passage re-ranking with BERT, in: arXiv preprint arXiv:1901.04085, 2019.
[16] Z. Dai, J. Callan, Deeper text understanding for IR with contextual neural language modeling, in: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR), 2019, pp. 985–988.